

AA2 — Proyecto integrador

Pipeline CI/CD para sistema de restaurante

David Alejandro Osorio Martínez

Gestión del software

Unidad 3 — Automatización y entrega continua

21/05/2026

Introducción

En este proyecto se diseñó y documentó un **pipeline de integración y entrega continua (CI/CD)** para el **sistema de gestión operativa de un restaurante**: la parte del software que usó el personal en sala y cocina para **administrar productos del menú, registrar pedidos y enviar comandas** al área de preparación (pantalla de cocina / KDS).

El alcance **no** incluyó reservas en línea ni pedidos desde la web del cliente; el enunciado del curso se tomó como referencia de negocio, y el pipeline se centró en el **flujo crítico de operación diaria** (donde un fallo en despliegue o en pruebas afectaba directamente el servicio).

Se justificó la automatización porque, sin pipeline, cada cambio dependía de pasos manuales (build, pruebas, subida al servidor), con riesgo de errores en producción y poca trazabilidad entre la versión desplegada y el commit probado.

1. Análisis del sistema

1.1 Funcionalidades principales (recorte documentado)

Área	Qué hacía en operación
Productos / menú	Alta y consulta de ítems que se vendían en sala (platos, bebidas, precios, agrupaciones).
Toma de pedidos	Captura del pedido en punto de venta: mesa, ítems, cantidades, formas de pago, cierre del documento de venta.
Comandas a cocina	Envío y seguimiento de ítems en pantalla de cocina (KDS): estados, prioridad por sección, coordinación con cocina.
Soporte transversal	Usuarios autenticados, inventario/bodega asociada al punto de venta, reportes de ventas del día.

1.2 Módulos incluidos en el pipeline

Se definieron estos **módulos** como candidatos a validación automática en cada integración:

1. **Catálogo de productos** — APIs y reglas de negocio del menú.
2. **Pedidos en sala** — creación y cierre de ventas desde el terminal táctil.
3. **Comandas (KDS)** — flujo cocina: ítems pendientes, marcado de preparación/entrega.
4. **Autenticación y permisos** — acceso de meseros y supervisores.

El **front** (pantalla táctil en navegador) se desplegó por separado; el pipeline documentado se enfocó en el **backend**, donde se concentró la lógica y las pruebas automatizadas.

1.3 Puntos críticos que se automatizaron

- **Integración de código** en rama de despliegue sin merge manual al servidor.
- **Pruebas unitarias** antes de empaquetar la imagen Docker.
- **Construcción reproducible** de la imagen del servicio API.

- **Publicación** en registro de contenedores con etiqueta por build.
- **Despliegue** en entorno de pruebas/staging por SSH, con `docker compose pull` (sin rebuild en el servidor).
- **Calidad estática** en integración continua (lint, formato, tipos).
- **Notificación** al equipo cuando una etapa falló o terminó.

2. Diseño del pipeline CI/CD

2.1 Etapas principales

Se organizó el flujo en **dos canales** complementarios:

Canal	Cuándo corría	Propósito
Integración continua (CI)	En cada <i>push</i> y <i>pull request</i>	Pruebas rápidas y revisión de estilo de código.
Entrega continua (CD)	En la rama <code>deploy-pipeline</code> (Azure Pipelines)	Build de imagen, push al registry y despliegue a servidor de pruebas.

Secuencia del pipeline de despliegue (CD):

Checkout → Pruebas (pytest en Docker) → Limpieza → Build imagen → Push registry
→ Despliegue SSH (pull + compose up) → Notificación por correo

Secuencia del pipeline de integración (CI en GitHub Actions):

Checkout → Build Docker → Job "test": migrate + pytest
→ Job "code-quality": flake8, black, isort, pylint, mypy

2.2 Herramientas por etapa

Etapas	Herramienta
Control de versiones	Git (repositorio del backend del restaurante)
Orquestación CD	Azure Pipelines (<code>azure-pipelines.yml</code>)
Orquestación CI	GitHub Actions (<code>.github/workflows/ci.yml</code>)
Contenedores	Docker + Docker Compose (<code>local.yml</code> , <code>testing.yml</code>)
Pruebas	pytest , Django migrate en entorno <code>test</code>
Calidad de código	flake8 , black , isort , pylint , mypy
Registro de imágenes	DigitalOcean Container Registry
Despliegue remoto	SSH + script remoto de deploy + docker compose
Base de datos en pruebas	PostgreSQL (contenedor en CI)

2.3 Diagrama del pipeline (despliegue)

flowchart LR

```
A[Commit en rama deploy] --> B[Checkout]
B --> C[Pruebas pytest]
```

```

C --> D[Build imagen Docker]
D --> E[Push al registry]
E --> F[SSH servidor staging]
F --> G[docker compose pull/up]
G --> H[Notificación correo]

```

2.4 Ejemplo de configuración (fragmento YAML — CD)

El pipeline se definió en Azure DevOps. A continuación se muestra un fragmento **simplificado** (sin secretos):

```

trigger:
  branches:
    include: [deploy-pipeline]

pool:
  vmImage: ubuntu-latest

variables:
  RUN_UNIT_TESTS: true
  DEPLOY_USE_REGISTRY_IMAGES: true

steps:
  - checkout: self

  - script: |
      # Pruebas: postgres + build django + pytest
      docker compose -f local.yml up -d postgres
      docker compose -f local.yml build django
      docker compose -f local.yml run --rm django bash -c "
        python manage.py migrate --settings=config.settings.test &&
        python -m pytest ewoa/ --create-db
      "
      displayName: "Pruebas unitarias"
      condition: ne(variables['RUN_UNIT_TESTS'], 'false')

  - script: |
      docker build -f compose/testing/django/Dockerfile -t REGISTRY/imagen:${(Build.BuildId)} .
      docker push REGISTRY/imagen:${(Build.BuildId)}
      displayName: "Build y push imagen"

  - script: |
      # SSH: pull de imagen y levantar servicios en servidor de pruebas
      bash scripts/deploy-remoto.sh
      displayName: "Despliegue staging"

```

2.5 Ejemplo de configuración (fragmento — CI en pull request)

```

name: CI restaurante-backend
on: [push, pull_request]
jobs:
  test:

```

```

runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - run: docker compose -f local.yml up -d postgres
  - run: docker compose -f local.yml run --rm django pytest ewoa/
code-quality:
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - run: docker compose run --rm django flake8 ewoa/ config/

```

3. Ejecución del pipeline y evidencias

3.1 Escenario exitoso (referencia)

Se documentó el recorrido esperado cuando todas las etapas pasaban:

1. Un desarrollador subió cambios en el módulo de pedidos a la rama **deploy-pipeline**.
2. Azure realizó el checkout del commit y levantó PostgreSQL en el agente.
3. **Pruebas:** migraciones de test y **pytest** sobre el paquete de aplicación → resultado **exitoso** (TESTS_RESULT=exitoso).
4. **Build:** **docker build** con Dockerfile de testing → imagen etiquetada con Build.BuildId.
5. **Push:** imagen publicada en el registry (BUILD_RESULT=exitoso).
6. **Deploy:** por SSH se copiaron manifiestos compose y script remoto; en el servidor se ejecutó **docker compose pull** y **up -d** del servicio API → DEPLOY_RESULT=exitoso.
7. **Correo:** resumen con estado por etapa y enlace al build en Azure DevOps.

En staging, la API quedaba disponible con la nueva imagen; la pantalla táctil de sala podía tomar pedidos y enviar comandas si el front apuntaba a ese entorno.

3.2 Evidencias de ejecución

Figura 1 — Vista del pipeline en Azure DevOps (ejecución global)

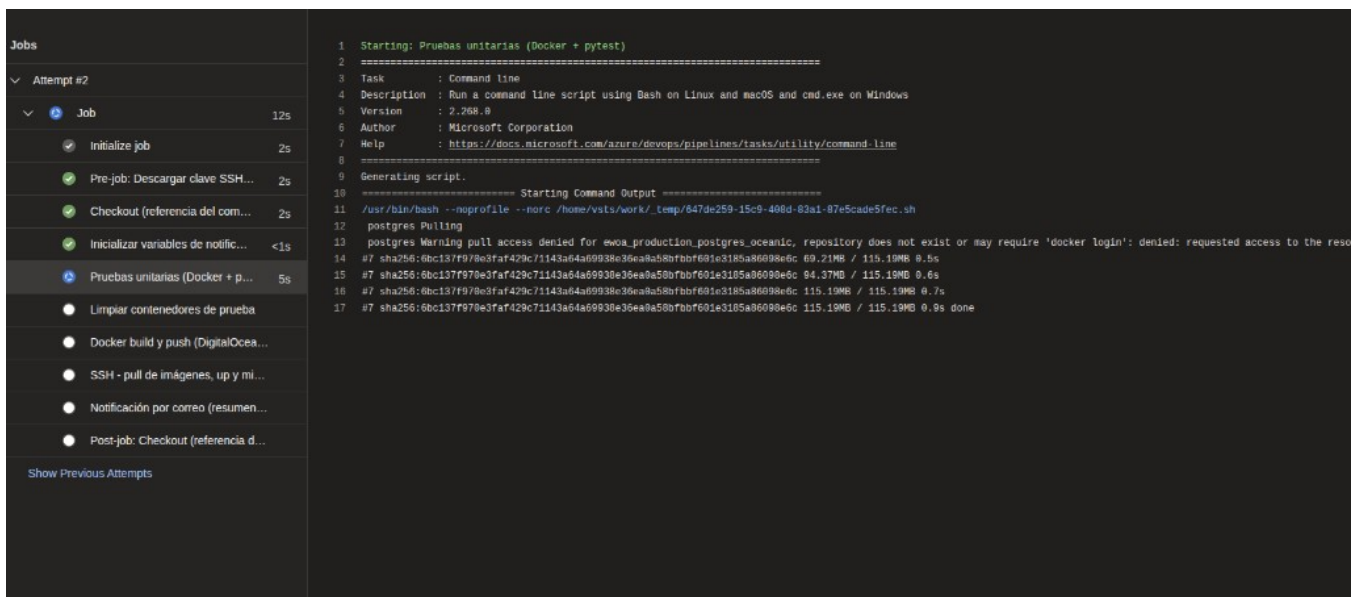


Figura 1: Ejecución del pipeline en Azure DevOps

Figura 2 — Checkout del commit en Azure (obtención del código)

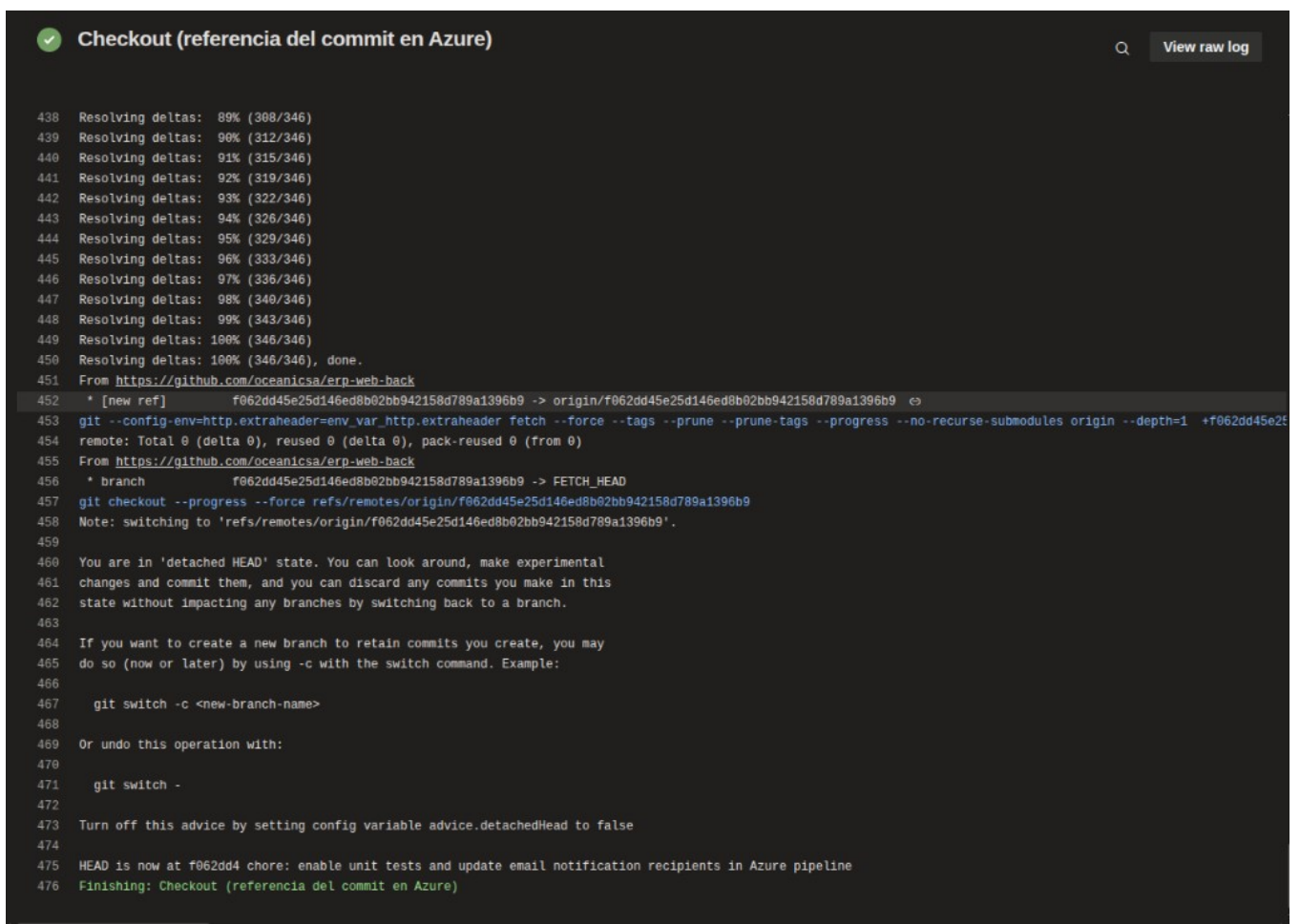
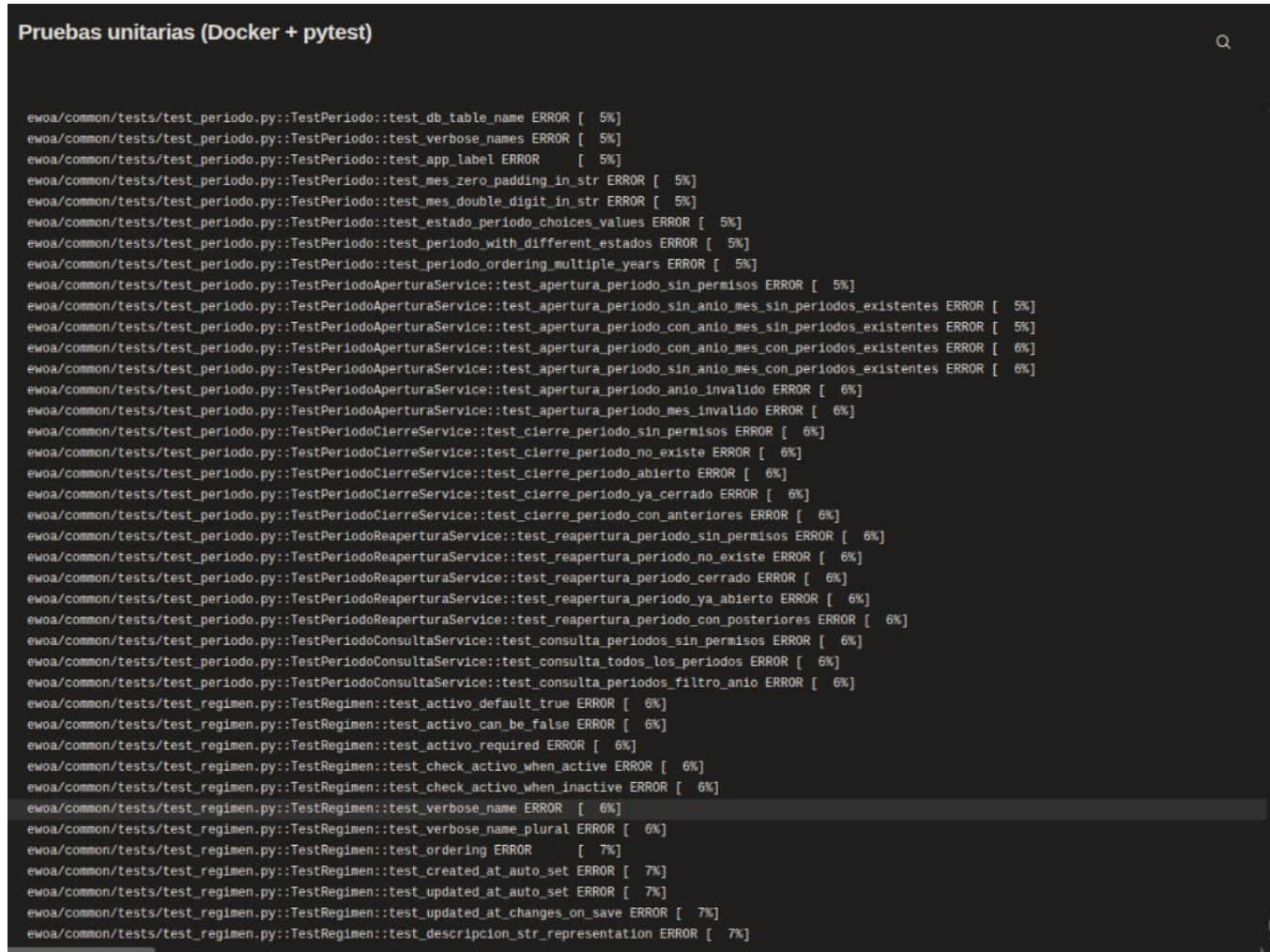


Figura 2: Checkout Azure

Figura 3 — Etapa de pruebas unitarias (step completo: Docker + pytest)



```
Pruebas unitarias (Docker + pytest)

ewoa/common/tests/test_periodo.py::TestPeriodo::test_db_table_name ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodo::test_verbose_names ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodo::test_app_label ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodo::test_mes_zero_padding_in_str ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodo::test_mes_double_digit_in_str ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodo::test_estado_periodo_choices_values ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodo::test_periodo_with_different_estados ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodo::test_periodo_ordering_multiple_years ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodoAperturaService::test_apertura_periodo_sin_permisos ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodoAperturaService::test_apertura_periodo_sin_anio_mes_sin_periodos_existentes ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodoAperturaService::test_apertura_periodo_con_anio_mes_sin_periodos_existentes ERROR [ 5%]
ewoa/common/tests/test_periodo.py::TestPeriodoAperturaService::test_apertura_periodo_con_anio_mes_con_periodos_existentes ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoAperturaService::test_apertura_periodo_sin_anio_mes_con_periodos_existentes ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoAperturaService::test_apertura_periodo_anio_invalido ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoAperturaService::test_apertura_periodo_mes_invalido ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoCierreService::test_cierre_periodo_sin_permisos ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoCierreService::test_cierre_periodo_no_existe ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoCierreService::test_cierre_periodo_abierto ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoCierreService::test_cierre_periodo_ya_cerrado ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoCierreService::test_cierre_periodo_con_anteriores ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoReaperturaService::test_reapertura_periodo_sin_permisos ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoReaperturaService::test_reapertura_periodo_no_existe ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoReaperturaService::test_reapertura_periodo_cerrado ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoReaperturaService::test_reapertura_periodo_ya_abierto ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoReaperturaService::test_reapertura_periodo_conposteriores ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoConsultaService::test_consulta_periodos_sin_permisos ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoConsultaService::test_consulta_todos_los_periodos ERROR [ 6%]
ewoa/common/tests/test_periodo.py::TestPeriodoConsultaService::test_consulta_periodos_filtro_anio ERROR [ 6%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_activo_default_true ERROR [ 6%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_activo_can_be_false ERROR [ 6%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_activo_required ERROR [ 6%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_check_activo_when_active ERROR [ 6%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_check_activo_when_inactive ERROR [ 6%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_verbose_name ERROR [ 6%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_verbose_name_plural ERROR [ 6%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_ordering ERROR [ 7%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_created_at_auto_set ERROR [ 7%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_updated_at_auto_set ERROR [ 7%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_updated_at_changes_on_save ERROR [ 7%]
ewoa/common/tests/test_regimen.py::TestRegimen::test_descripcion_str_representation ERROR [ 7%]
```

Figura 3: Ejecución de pruebas unitarias

El step terminó su ejecución; pytest reportó errores en la suite (el pipeline marcó la etapa como fallida y no continuó con build ni despliegue).

Figura 4 — Verificación de build del front (GitHub Actions: step “Build project”)

```

257 dist/assets/ConfiguracionUsuarioView-6CcgHE5Q.js 27.99 kB | gzip: 8.45 kB
258 dist/assets/documentos-D-kMvHT-.js 28.67 kB | gzip: 6.71 kB
259 dist/assets/ResolucionesView-DRVpyCXe.js 30.51 kB | gzip: 9.03 kB
260 dist/assets/KdsView-DN1x2DU7.js 30.61 kB | gzip: 9.64 kB
261 dist/assets/LogsView-CJ52jcz2.js 31.28 kB | gzip: 7.65 kB
262 dist/assets/ReciboCajaEgresoView-CbLSzV1m.js 35.04 kB | gzip: 10.42 kB
263 dist/assets/ReciboCajaIngresoView-Ch3wutr8.js 35.30 kB | gzip: 10.44 kB
264 dist/assets/ConfiguracionesView-C0f7W0tg.js 40.20 kB | gzip: 10.30 kB
265 dist/assets/CxpInformesView-2KxrOhjJ.js 44.54 kB | gzip: 9.47 kB
266 dist/assets/ConfiguracionGeneralView-DAnw926b.js 45.12 kB | gzip: 10.70 kB
267 dist/assets/CxcInformesView-4Jv8yqPY.js 49.51 kB | gzip: 10.55 kB
268 dist/assets/TercerosModalView-B0spDDJc.js 52.84 kB | gzip: 14.80 kB
269 dist/assets/SvgIcon-BwIHvovV.js 52.95 kB | gzip: 16.39 kB
270 dist/assets/TiposDocumentosInvView-C9DTbxoD.js 60.21 kB | gzip: 14.62 kB
271 dist/assets/ListadoDeRecursos-D-m7n0Et.js 61.58 kB | gzip: 16.51 kB
272 dist/assets/TerceroFormulario-CnM8V7P9.js 62.69 kB | gzip: 16.15 kB
273 dist/assets/CxcTransaccionesView-BLkgB1Uu.js 64.26 kB | gzip: 18.44 kB
274 dist/assets/CxpRegistrarPagoView-wQ16jCQ5.js 66.09 kB | gzip: 17.38 kB
275 dist/assets/UsuariosView-Crbke-vu.js 69.12 kB | gzip: 17.80 kB
276 dist/assets/SuscripcionView-grXaDdg1.js 72.53 kB | gzip: 21.12 kB
277 dist/assets/ModalContratosAIU-DBkC2uPc.js 83.02 kB | gzip: 19.26 kB
278 dist/assets/ProductosView-BrjFmVeu.js 83.79 kB | gzip: 22.50 kB
279 dist/assets/GenericAutoComplete-DFLozzxN.js 86.27 kB | gzip: 31.54 kB
280 dist/assets/PosTouchScreenView-2413YthX.js 91.40 kB | gzip: 27.12 kB
281 dist/assets/ContenedorMovimiento-B4Q2c8sh.js 92.48 kB | gzip: 24.99 kB
282 dist/assets/MovimientoGeneric-D2eWl7fL.js 108.26 kB | gzip: 28.67 kB
283 dist/assets/FacturacionInformesView-uzpPqGH0.js 113.38 kB | gzip: 22.33 kB
284 dist/assets/MovimientoGenericDisenoB-CLiAPImN.js 115.57 kB | gzip: 33.55 kB
285 dist/assets/PosInformesView-gJ38jyeQ.js 137.85 kB | gzip: 26.88 kB
286 dist/assets/HabilitacionDianView-BhXSQE-V.js 419.99 kB | gzip: 110.61 kB
287 dist/assets/index-CfZ067gX.js 937.24 kB | gzip: 395.17 kB
288
289 (!) Some chunks are larger than 500 kB after minification. Consider:
290 - Using dynamic import() to code-split the application
291 - Use build.rollupOptions.output.manualChunks to improve chunking: https://rollupjs.org/configuration-options/#output-manualchunks
292 - Adjust chunk size limit for this warning via build.chunkSizeWarningLimit.
293 ✓ built in 27.11s
294
295 Finishing: Build project

```

Figura 4: Build del proyecto front

Se compiló con Vite (*built in 27.11s*); el bundle incluyó módulos de productos, POS e informes. En el pipeline de despliegue del backend, la etapa equivalente fue **docker build + push** al registry.

3.2.1 Pasos en el front (punto de venta y cocina)

Se registró el flujo operativo que consumió el backend desplegado por el pipeline:

Paso	Acción	Módulo
1	Selección de mesa o zona en sala	Pedidos
2	Búsqueda de productos, armado del carrito y datos del documento	Productos + pedidos
3	Cobro (formas de pago, propina, confirmación)	Pedidos
4	Comanda en cocina — estado Pendiente (botón <i>Comandar</i> / KDS)	Comandas
5	Seguimiento en KDS — estado Preparada y cierre (<i>Completar</i>)	Comandas

Figura 5 — Paso 1: selección de mesa

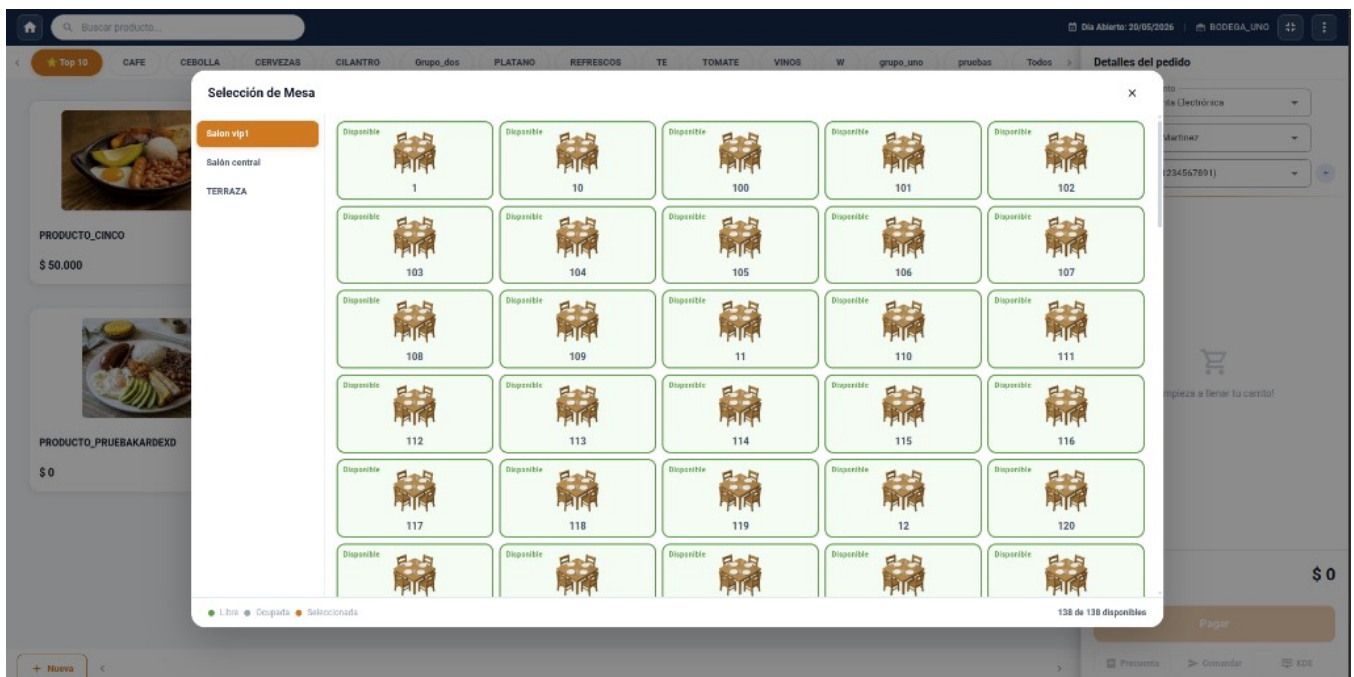


Figura 5: Selección de mesa

Figura 6 — Paso 2: toma de pedido (productos en carrito)

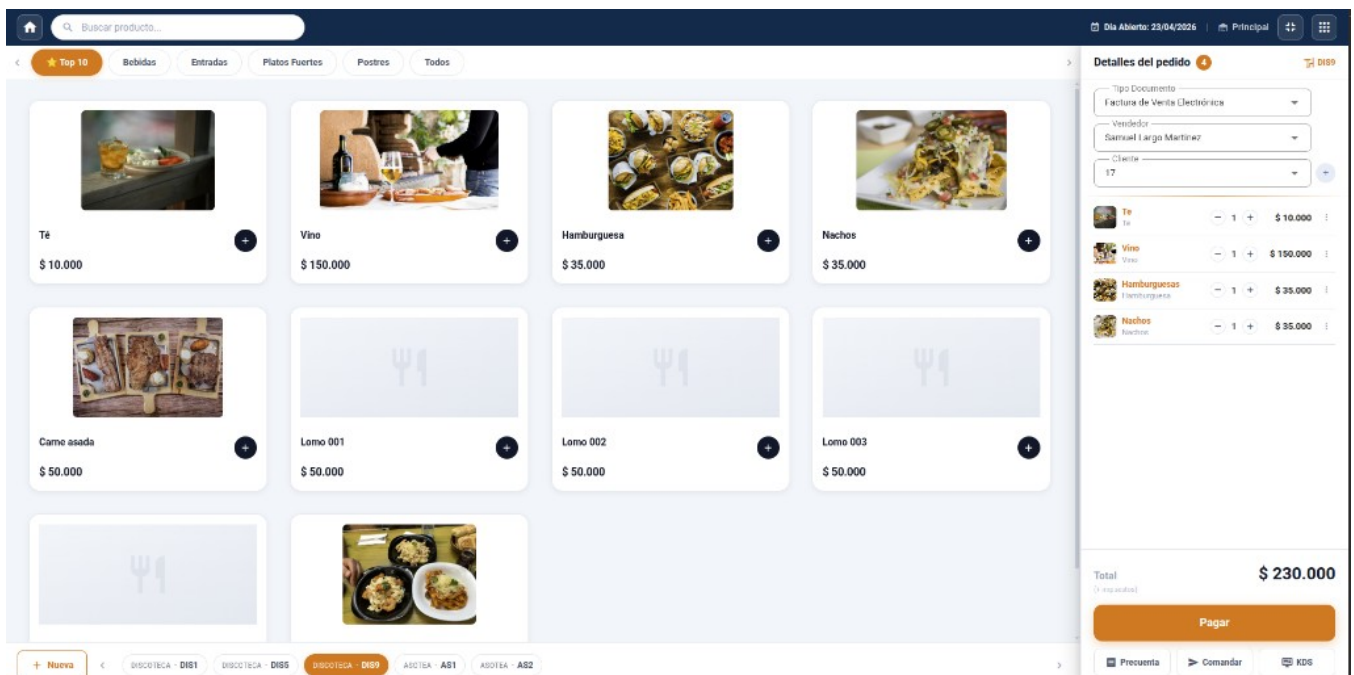


Figura 6: Toma de pedido

Figura 7 — Paso 3: procesar pago

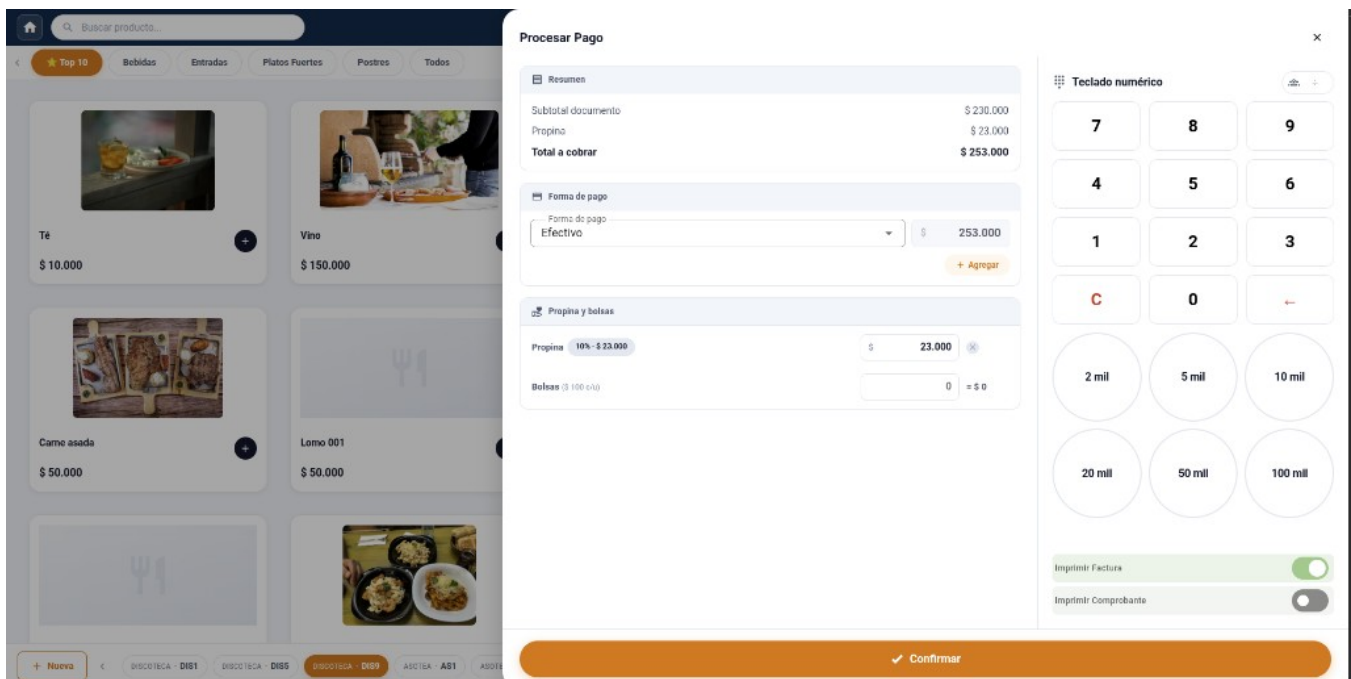


Figura 7: Procesar pago

Figura 8 — Paso 4: comanda en cocina (KDS — pendiente)

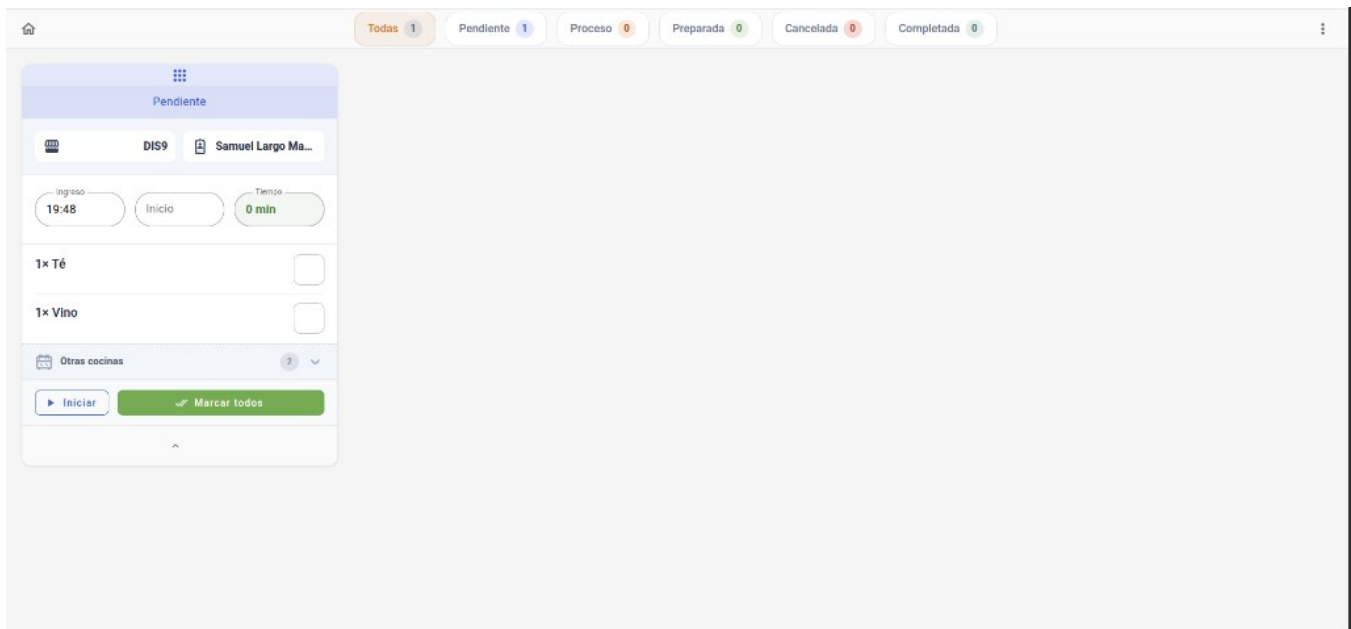


Figura 8: KDS pendiente

Figura 9 — Paso 5: comanda preparada (KDS)

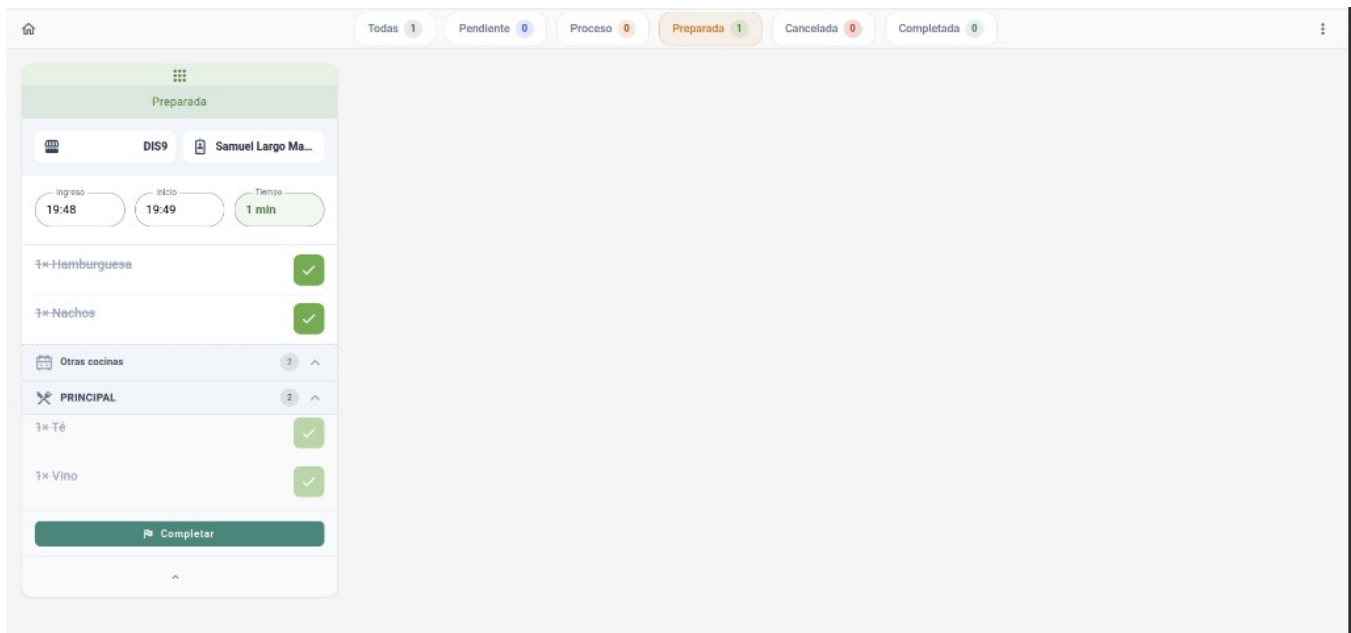


Figura 9: KDS preparada

Figura 10 — Integración continua en GitHub Actions (workflow lint.yml en pull request)

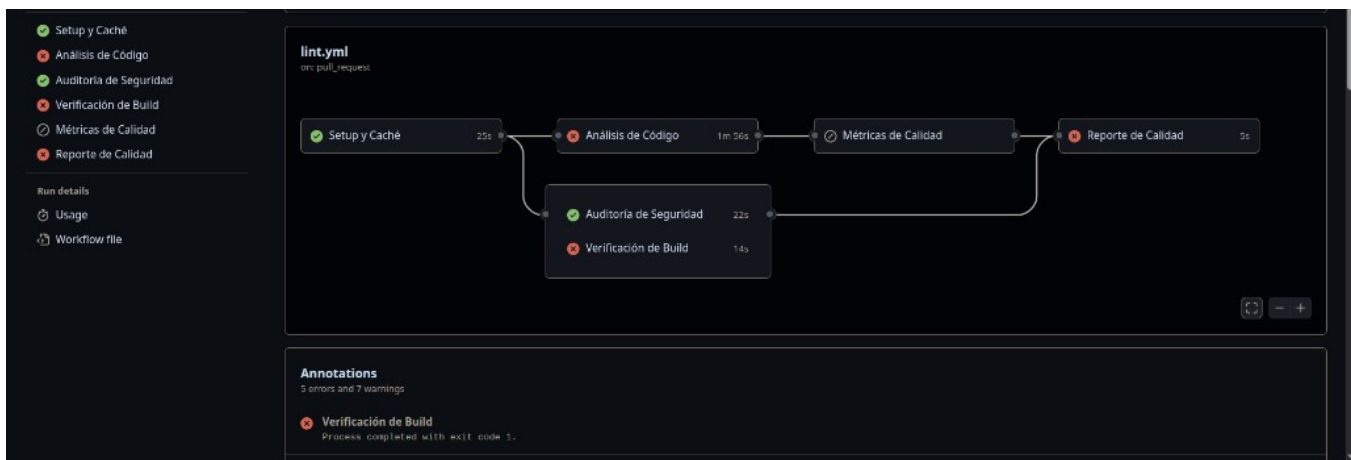


Figura 10: GitHub Actions CI

Los jobs de análisis de código y verificación de build fallaron en la corrida capturada; el pipeline detuvo el reporte final (gate de calidad antes de desplegar).

3.3 Fallo observado en ejecución real y corrección

Evidencia (Figura 3): en una corrida del pipeline, el step de **pytest** finalizó con múltiples **ERROR** en tests de módulos transversales; la etapa quedó en fallo.

Etapa	Resultado
Pruebas unitarias	Fallido
Build imagen	Omitido (no corrió al fallar las pruebas)

Etapa	Resultado
Despliegue Correo	Omitido Asunto con etiqueta FALLIDO y detalle Pruebas unitarias [FALLO]

Medidas de corrección que se propusieron:

1. Revisar el log de pytest en Azure y localizar el test roto.
2. Corregir la regla de negocio o el test; ejecutar `docker compose run django pytest` en local.
3. Abrir un PR y esperar **GitHub Actions** en verde (CI).
4. Fusionar y relanzar el pipeline de despliegue hasta obtener el step de pruebas en verde.

3.4 Comandos de consola (referencia local)

```
# Entorno de prueba local (mismo criterio que el pipeline)
docker compose -f local.yml up -d postgres
docker compose -f local.yml build django
docker compose -f local.yml run --rm django bash -c \
  "python manage.py migrate --settings=config.settings.test && python -m pytest ewoa/pos/ -v"
```

4. Validación de calidad y seguridad

En el pipeline se contemplaron requisitos **no funcionales** de la siguiente manera:

Requisito	Cómo se validó en el pipeline
Calidad de código	Job de flake8, black, isort, pylint, mypy en CI.
Confiabilidad funcional	pytest con base de datos de prueba aislada.
Reproducibilidad del despliegue	Imagen Docker versionada por Build.BuildId; el servidor solo hizo pull.
Disponibilidad del despliegue	Script remoto con healthcheck implícito vía compose; reintento manual documentado.
Seguridad operativa	Clave SSH en <i>Secure Files</i> de Azure; token de registry como secreto; sin build en producción.

Dos técnicas/herramientas que se integraron:

1. **Análisis estático de código** — pylint + flake8 en el job code-quality.
2. **Pruebas automatizadas de regresión** — pytest sobre módulos de pedidos, POS y comandas antes de permitir el build.

(El escaneo de vulnerabilidades en imagen, por ejemplo Trivy, quedó registrado como mejora futura.)

5. Buenas prácticas y escalado

Buenas prácticas que se aplicaron o documentaron:

- Integración en rama dedicada antes de producción.
- Pruebas automáticas como *gate* antes del build.
- Imagen inmutable por build (sin “git pull + build” en el servidor).
- Separación CI (cada PR) frente a CD (rama de despliegue).
- Notificación al equipo al finalizar el pipeline.
- Limpieza de contenedores de prueba (`docker compose down`) aunque fallara un paso intermedio.

Escalado que se sugirió:

- Añadir **entorno de producción** con aprobación manual (*approval gate*) en Azure.
- Desplegar el **front** en CDN o bucket estático en un job paralelo.
- Incorporar **Trivy** o similar en la etapa de build para vulnerabilidades.
- Pruebas de **carga** ligeras en staging tras el deploy (módulo de pedidos en hora pico simulada).
- **Blue/green** o rolling update si el restaurante escaló a varias sucursales.

Conclusiones

Se documentó un pipeline **CI/CD** para el backend del sistema operativo del restaurante: automatizó pruebas, empaquetado y despliegue a staging, alineado con prácticas DevOps del curso. La relación con la **calidad del producto** fue directa: un pedido mal validado o un deploy roto se detectó **antes** de que el personal de sala lo sufriera en producción.

Aprendizajes obtenidos: (1) CI y CD cumplieron roles distintos pero complementarios; (2) Docker y el registry permitieron despliegues repetibles; (3) fallar las pruebas temprano evitó incidentes en cocina y caja; (4) documentar variables y secretos del pipeline formó parte del mantenimiento del sistema.

Referencias

- Documentación Azure Pipelines — YAML schema y tasks.
- Docker Compose — orquestación local y remota.
- pytest — pruebas automatizadas en Python/Django.